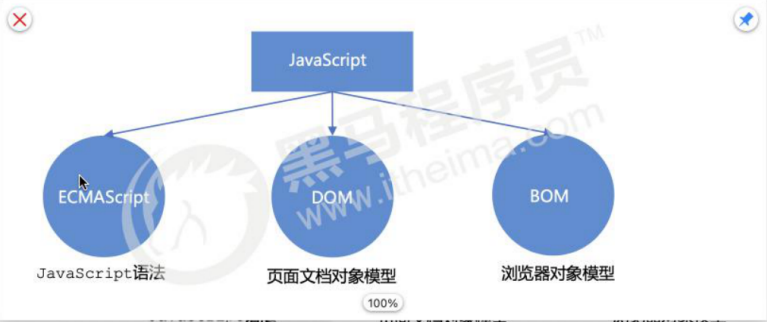


内部标签

```
<script>
    alert('弹出警示框！！！');
</script>
```

外部引入

src中添加js的地址

```
<script src="./bootstrap-3.4.1-dist/js/bootstrap.js"></script>
```

script最好放在body'的后面因为浏览器解析是从上往下解析先解析css和html再解析js

```
alert('弹出警示框！！！');//警示
console.log("khhhh");//在控制台输出
prompt('请输入内容')//输入框
```

变量

存储数据的容器 es5使用var es6使用let定义变量

变量声明和赋值 ‘=’ 为赋值符号；

```
var sex='Nan';
```

//变量可以声明不赋值也可以声明并且赋值 变量声明未赋值值为undefind

变量未声明直接使用会报错

变量重新赋值会覆盖之前创建的变量

变量连写直接添加逗号即可，新手容易出错，不建议

取名避开name ， name为全局变量

数据类型

string 字符串

number 数字 （整数，浮点数）

boolean 布尔 （true，flase）

undefined 未定义

null 空值

symbol

进制转换

2-10进制 8进制 16进制之间的转换

number

infinity 正值为正无穷大，负的为负无穷大

NAN为非数字但属于number数据类型

isNaN()用来判断值，是否是一个非数字（比较特殊）

string 字符串

单双引号都可以，一般用单引号

引号嵌套时候可以外双内单或者内双外单

转义符

\n 换行

\\ 斜杠

\' 单引号

\t tab

\b 空格

空字符串

var srt ='';

字符串的拼接

拼接两个字符串中间添加 + 号

数字和字符串拼接，会转成字符串

boolean 布尔值

布尔值通常以 is has should开头

flalse》》0 true》》1

undefind

声明一个变量不赋值默认就是undefind

null

null只有一个值null，表示一个空对象

symbol

es6新增类型

每个symbol（）中返回的值是唯一的可以作为对象里的键

typeof（）数据类型检测

数据转换

转number

prompt（）输入的数据为字符型 （无法转换时会转换为NaN）

parseint（） 转换为整数

parsenumber（） 转换为浮点

number （）强制转换

隐式转换

转string

.tostring（）转string

string()强制转换

转Boolean

0 NAN null ‘’ undefined 会转成false

运算

正常 数学运算即可
取模就是取余，%表示
前置递增

```
var rem=rem+1
    简写   ++rem
```

后置递增

```
  var rem=rem+1
      rem++   后置加
```

前置递增和后置递增的区别
前置递增先加1再赋值，后置递增先赋值再加1

比较运算符
> 大于 < 小于 >= 大于等于 <=小于等于 == 等于（不比较数据类型） !==不等于 === 全等于（会比较数据类型）

逻辑运算符号
&&与（并且）： 两个条件同时为true
||或： 只要有一个条件满足就为true
！ 非 取反

短路运算
&&左边有一个为false 右边值不会参与比较
||左边如果为true 右边不会运行

赋值运算的简写
+=

```
let num =num+5
    num+=5
```

+= -= /= *= %=

运算符的优先级

. () []
++ –
算术运算符
关系运算符
逻辑运算符（&& ||）
三目运算符
赋值运算符（= += *= /= != -= ）

分支语句

if后面只有一句话的时候可以写成一行并省略后面的{}

```
if( 条件){
            语句
        }else{
                语句
        }if
        else if(条件){
          语句
            }
```

三元表达式

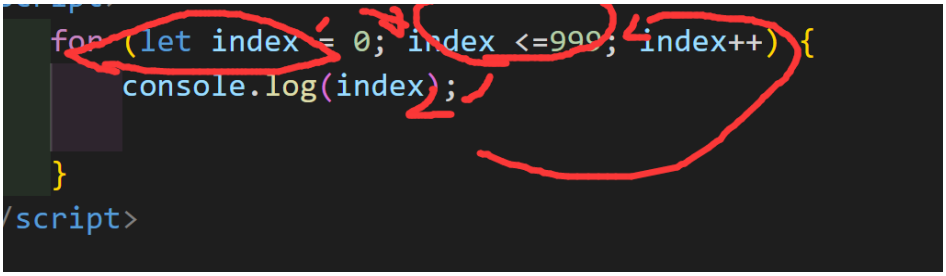
表达式1（条件）？表达式2：表达式3

switch case

```
let score = prompt('fes')
        switch (true) {
            case score >= 90:
                console.log("优秀");
                break;
            case score >= 80 && score < 90:
                console.log("良好");
                break;
            case score >= 60 && score < 80:
                console.log("及格");
                break;
            default:
                console.log('不及格');
                break;
        }
```

循环

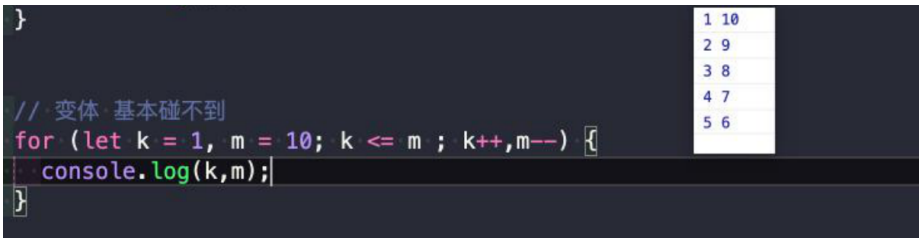
用来将重复的事情简单化




```
for (let index = 0; index <=999; index++) {
            console.log(index);

        }
```

变体：



死循环：

不写条件，会照成死循环，但是在中间添加条件可以直接停止

do while循环

```
do {

        } while (condition);
```

white 循环

```
 while (condition) {

        }
```

continue跳出本次循环，循环继续

```
continue
```

break 直接跳出循环，后面的循环停止

```
break
```

数组

数组可以放任意类型的数据

```
let     name =[1,2,3,4, true ,]
```

通过对象创建

```
let list=  new Array(1,2,3,4,)
```

数组的访问

数组访问是通过数组的索引访问，编号从0开始

```
   console.log(lins[5]);
```

数组的遍历

在循环中遍历数组

```
for (let i = 0; i < arr.length; i++) {

            i==arr[i].length-1?su+=arr[i]+' ': su+=arr[i]+'|'

         }
```

新增数组元素

push（）方法

push方法可以向数组追加一个数据

通过length属性添加数组长度

函数

自定义函数:

函数的作用为了封装重复的代码块

function 定义函数 后面函数的名字（）{ 方法体}

```
function hanshuming (){
            //函数体
```

```
        }
```

给函数添加参数有两个（形参，实参）直接写入函数的括号中

形参

形式参数，函数中的变量来自新式参数，后期可以改变形参中的值，
来改变最后的结果

实参

实际传递的参数，再函数调用的时候使用加入调用时候的括号中，实参对应形参 多个值要传输用 ‘，号隔开
里面的值按照顺序一 一对应，改变位置参数不同，如果实际参数小于实参，就按照顺序匹配

```
function data( num1,num2){//形参

        }
        data(1,2)//传给形参数，等价于 num1=1 num2=2
```

return ： 返回函数值

return可以结束函数的执行，后面的代码不执行
返回只能返回一个值，但是可以转成对象和数组返回这样就可以返回多个值

arguments

arguments可以接受函数的参数（形参）
arguments 是一个对应于传递给函数的参数的类数组对象。（理解为伪数组）es6代替

通过表达式创建函数（匿名函数）

```
 var  fn =function(){

        }
```

递归函数 （函数自身调用自身）

要给一个停止的条件不然会一直递归下去，浏览器会崩溃

```
function test1(){
            console.log(dd);
            test1()
```

```
}
```

函数的作用域

全局作用域

- 最外层函数 和在最外层函数外面定义的变量拥有全局作用域
- 少用全局变量1.变量名会污染

函数作用域

只会在函数中作用，超出函数不起作用函数与函数之间的变量不能共享

js预解析

jsdiamagnetic是由浏览器解释器运行，分为两步预解析和代码执行

JavaScript 代码是由浏览器中的 JavaScript 解析器来执行的。JavaScript 解析器在运行 JavaScript 代码的时候分为两步：预解析和代码执行。

- **预解析：** 在当前作用域下, JS 代码执行之前，浏览器会默认把带有 var 和 function 声明的变量在内存中进行提前声明或者定义。
- **代码执行：** 从上到下执行JS语句。

预解析只会发生在通过 var 定义的变量和 function 上。学习预解析能够让我们知道为什么在变量声明之前访问变量的值是 undefined，为什么在函数声明之前就可以调用函数。

黑马程序员 www.itheima.com